Lab 08

Objective:

The objective of this lab is to learn how to implementat recurrent neural networks in keras.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement RNN model in Keras.
- Train RNN on a dataset in Keras.

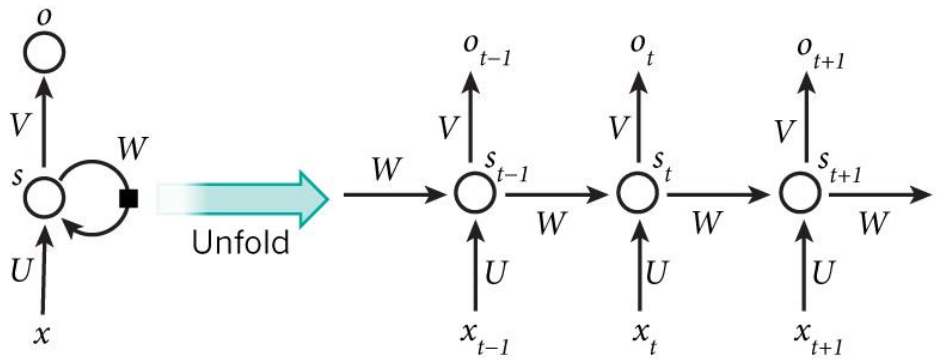
Instructor Note:

As pre-lab activity, read and implement the following tutorial:

1. https://machinelearningmastery.com/understanding-simple-recurrent-neural-networks-in-keras/
2. https://www.tensorflow.org/guide/keras/working_with_rnns

20) Useful Concepts

- **Recurrent Neural Networks (RNNs)** are a family of networks that are suitable for learning representations of sequential data like text in Natural Language Processing (NLP). The idea behind RNNs is to make use of sequential information. In a traditional neural network we assume that all inputs (and outputs) are independent of each other. But for many tasks that is a very bad idea. If we want to predict the next word in a sentence we better know which words came before it. RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being depended on the previous computations. Another way to think about RNNs is that they have a "memory" which captures information about what has been calculated so far. In theory RNNs can make use of information in arbitrarily long sequences, but in practice they are limited to looking back only a few steps. Here is what a typical RNN looks like:



  - The above diagram shows a RNN being unrolled (or unfolded) into a full network.
  - By unrolling we simply mean that we write out the network for the complete sequence.
  - For example, if the sequence we care about is a sentence of 5 words, the network would be unrolled into a 5-layer neural network, one layer for each word.

- **RNN Computations:**
  - We can describe the computations within an RNN in terms of equations.
  - The internal state of the RNN at a time t is given by the value of the hidden vector ht, which is the sum of the product of the weight matrix W and the hidden state ht-1 at time t-1 and the product of the weight matrix U and the input xt at time t, passed through the tanh nonlinearity.
  - The choice of tanh over other nonlinearities has to do with its second derivative decaying very slowly to zero.
  - This keeps the gradients in the linear region of the activation function and helps combat the vanishing gradient problem.

- The output vector yt at time t is the product of the weight matrix V and the hidden state ht, with softmax applied to the product so the resulting vector is a set of output probabilities:

$$ht = tanh(Wht - 1 + UXt)$$

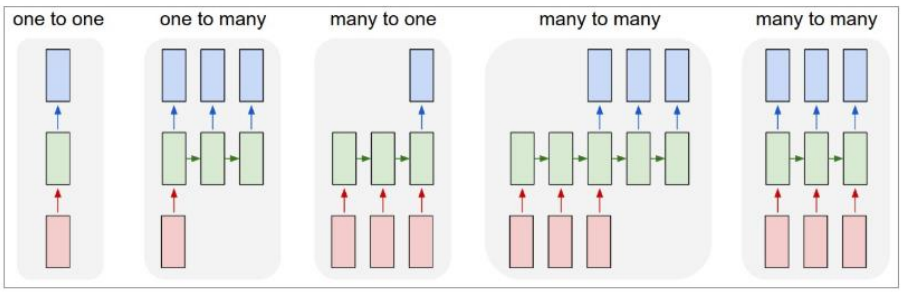
$$yt = softmax(Vht)$$

- Keras provides the SimpleRNN (for more information refer to: https://keras.io/layers/recurrent/) recurrent layer that incorporates all the logic we have seen so far, as well as the more advanced variants such as LSTM and GRU that we will see later in this kernel.
- **SimpleRNN cells:**
  - Traditional multilayer perceptron neural networks make the assumption that all inputs are independent of each other. This assumption breaks down in the case of sequence data.
  - Time series data, such as stock prices, also exhibit a dependence on past data, called the secular trend.
  - RNN cells incorporate this dependence by having a hidden state, or memory, that holds the essence of the past.
  - The value of the hidden state at any point in time is a function of the value of the hidden state at the previous time step and the value of the input at the current time step, that is:

$$ht = \emptyset(h_{t-1}.X_t)$$

  - h_t and h_{t-1} are the values of the hidden states at the time steps t and t-1 respectively, and x_t is the value of the input at time t.
  - The above equation is recursive, that is, ht-1 can be represented in terms of h_{t-2} and x_{t-1}, and so on, until the beginning of the sequence.
  - This is how RNNs encode and incorporate information from arbitrarily long sequences.
- **RNN topologies**

- The APIs for MLP and CNN architectures are limited. Both architectures accept a fixed-size tensor as input and produce a fixed-size tensor as output; and they perform the transformation from input to output in a fixed number of steps given by the number of layers in the model.
- RNNs don't have this limitation—you can have sequences in the input, the output, or both. This means that RNNs can be arranged in many ways to solve specific problems.
- RNNs combine the input vector with the previous state vector to produce a new state vector. This can be thought of as similar to running a program with some inputs and some internal variables.
- The RNNs are more exciting because they allow us to operate over sequences of vectors: Sequences in the input, the output, or in the most general case both.
- This property of being able to work with sequences gives rise to a number of common topologies shown below:



- In the above diagram each rectangle is a vector and arrows represent functions (e.g. matrix multiply).

- Input vectors are in red, output vectors are in blue and green vectors hold the RNN's state.
- From left to right:
    - Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification).
    - Sequence output (e.g. image captioning takes an image and outputs a sentence of words).
    - Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment).
    - Sequence input and sequence output (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French).
    - Synced sequence input and output (e.g. video classification where we wish to label each frame of the video).

- **Vanishing and exploding gradients**
    - Just like traditional neural networks, training the RNN also involves backpropagation.

- The difference in this case is that since the parameters are shared by all time steps, the gradient at each output depends not only on the current time step, but also on the previous ones.
- This process is called backpropagation through time (BPTT).
- Regular RNNs might have a difficulty in learning long range dependencies.
- This kind of dependencies between sequence data is called long-term dependencies because the distance between the relevant information and the point where it is needed to make a prediction is very wide.
- As this distance becomes wider, RNNs have a hard time learning these dependencies because it encounters either a vanishing or exploding gradient problem.
- These problems arise during training of a deep network when the gradients are being propagated back in time all the way to the initial layer.
- The gradients coming from the deeper layers have to go through continuous matrix multiplications because of the the chain rule, and as they approach the earlier layers, if they have small values (<1), they shrink exponentially until they vanish and make it impossible for the model to learn. This is the vanishing gradient problem.
- While on the other hand if they have large values (>1) they get larger and eventually blow up and crash the model. This is the exploding gradient problem.
- While there are a few approaches to minimize the problem of vanishing gradients, such as proper initialization of the W matrix, using a ReLU instead of tanh layers, and pre-training the layers using unsupervised methods, the most popular solution is to use the LSTM or GRU architectures (discussed next).
- These architectures have been designed to deal with the vanishing gradient problem and learn long term dependencies more effectively.

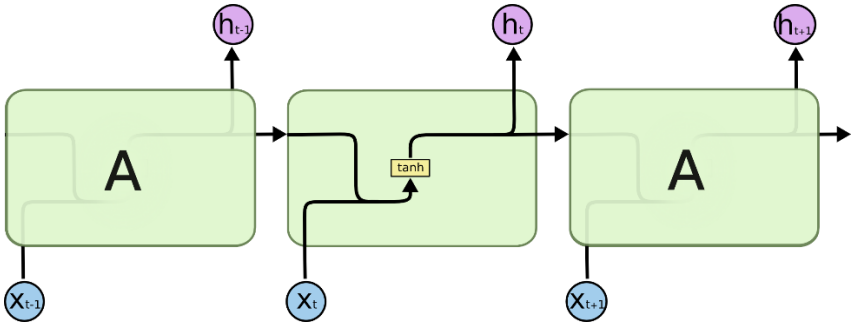
- **Long Short Term Memory (LSTM)**

- **Long Short Term Memory** networks – usually just called **LSTMs** – are a special kind of RNN, capable of learning long-term dependencies. They work tremendously well on a large variety of problems, and are now widely used.

- LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behaviour.

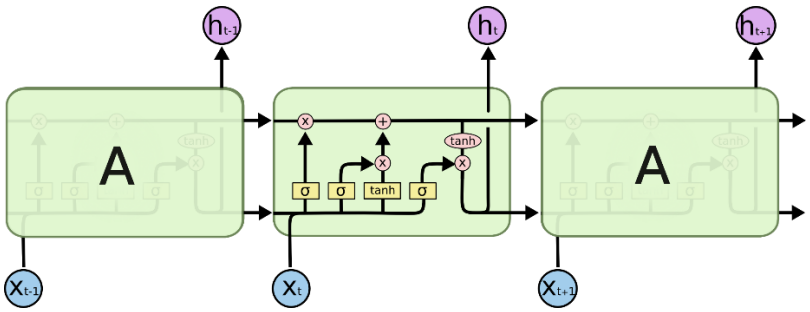
- All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

- The repeating module in a standard RNN contains a single layer as shown below:



- LSTMs also have this chain like structure, but the repeating module has a different structure.

- Instead of having a single neural network layer, there are four, interacting in a very special way as shown below:



21) Solved Lab Activites

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*60*	*Low*	*CLO-6*

Activity 1: Understand and implement the tutorial available at: https://machinelearningmastery.com/understanding-simple-recurrent-neural-networks-in-keras/

22) Graded Lab Tasks

The purpose of this lab task is to help students understand and manipulate different data structures in Python, specifically lists, dictionaries, and tuples. Students will perform various operations on these data structures, including creation, modification, and data access.

Lab Task 1: Implement a Movie Review Classifier using Simple RNN in Keras

Students will build and train a simple RNN for classifying movie reviews. For this task, use IMDb movie reviews from Stanford AI Lab's Large Movie Review Dataset.

Instructions for Submission:

13. Submit a Jupyter Notebook or Python script with the complete implementation.
14. Include comments in the code to explain each step.
15. Provide a brief report (1-2 pages) summarizing the following:
    - Data preprocessing steps
    - Model architecture and rationale for chosen hyperparameters
    - Training process and evaluation metrics
    - Observations and insights from the results